

Design Pattern ที่ใช้

1. Design Pattern ที่ใช้

ระบบ Promotion Engine ควรออกแบบเป็น **Rule-based Promotion Engine** โดยใช้หลาย Design Pattern ร่วมกัน เพื่อให้รองรับ promotion หลายแบบ เพิ่ม promotion ใหม่ได้ง่าย และไม่ทำให้ logic เดิมพัง

1. Strategy Pattern

ใช้สำหรับแยก logic การคำนวณ promotion แต่ละประเภทออกจากกัน

ตัวอย่าง:

Promotion Type	Strategy
ลดเป็นเปอร์เซ็นต์	PercentageDiscountStrategy
ลดเป็นจำนวนเงิน	FixedAmountDiscountStrategy
ลดทั้ง order	CartDiscountStrategy
ส่งฟรี	FreeShippingStrategy
ซื้อ X แถม Y	BuyXGetYStrategy

เหตุผลที่ใช้:

- แต่ละ promotion มีวิธีคำนวณไม่เหมือนกัน
- ไม่ควรเขียน `if/else` ใหญ่ๆ รวมทุก promotion ไว้ที่เดียว
- เวลาเพิ่ม promotion ใหม่ ให้เพิ่ม Strategy ใหม่ ไม่ต้องแก้ logic เดิมมาก

แนวคิดหลักคือ:

Promotion Engine ไม่ต้องรู้รายละเอียดว่าลดแบบไหน
Engine แค่เลือก Strategy ที่ถูกต้อง แล้วให้ Strategy นั้นคำนวณ

2. Factory / Registry Pattern

ใช้สำหรับเลือก Strategy ให้ตรงกับ `action_type` ของ promotion

ตัวอย่าง:

```
action_type = PERCENTAGE_DISCOUNT
→ ใช้ PercentageDiscountStrategy

action_type = FIXED_AMOUNT_DISCOUNT
→ ใช้ FixedAmountDiscountStrategy
```

เหตุผลที่ใช้:

- ทำให้ Promotion Engine ไม่ต้องผูกกับ class ของ promotion โดยตรง
- เพิ่ม promotion ใหม่ได้โดย register strategy ใหม่
- ลดการแก้ไข code เดิมใน calculation flow

3. Specification Pattern

ใช้สำหรับตรวจสอบว่า promotion นี้ "ใช้ได้หรือไม่" กับ order ปัจจุบัน

ตัวอย่าง condition:

Condition	ความหมาย
<code>PRODUCT_ID = 1</code>	ใช้กับสินค้า Product 1
<code>CATEGORY_ID IN (...)</code>	ใช้กับสินค้าบางหมวด
<code>MIN_ORDER_AMOUNT >= 1000</code>	ยอดสั่งซื้อขั้นต่ำ
<code>COUPON_CODE = SAVE100</code>	ต้องใช้ coupon
<code>DATE_RANGE</code>	อยู่ในช่วงเวลาที่กำหนด

เหตุผลที่ใช้:

- แยก logic ตรวจสอบ eligibility ออกจาก logic คำนวณ
- ทำให้ condition เพิ่มใหม่ได้ง่าย
- รองรับ condition หลายแบบ เช่น AND / OR

4. Chain / Pipeline Pattern

ใช้จัดลำดับขั้นตอนการคำนวณ promotion

ลำดับที่แนะนำ:

```
Validate Order
→ Load Product Price
→ Load Active Promotions
→ Check Conditions
→ Sort Promotions
→ Apply Item-level Promotion
→ Apply Cart-level Promotion
→ Apply Coupon Promotion
→ Apply Shipping Promotion
→ Rounding
→ Generate Breakdown
```

เหตุผลที่ใช้:

- ทำให้ flow การคำนวณชัดเจน
- ควบคุมลำดับ promotion ซ้อนกันได้
- Debug ง่าย เพราะรู้ว่าแต่ละ step ทำอะไร

5. Policy Pattern

ใช้จัดการ rule ของ promotion ซ้อนกัน เช่น stacking และ conflict

ตัวอย่าง policy:

Field	ความหมาย
priority	promotion ไหนคำนวณก่อน
stackable	ใช้ร่วมกับ promotion อื่นได้หรือไม่
exclusive	ถ้าใช้ promotion นี้แล้ว ห้ามใช้ promotion อื่นที่ชนกัน
stop_processing	ถ้าใช้ promotion นี้แล้ว ให้หยุดคำนวณ promotion ถัดไป

เหตุผลที่ใช้:

- ป้องกันการลดราคาซ้ำผิดพลาด
- ทำให้ business rule อยู่ใน config/table
- เปลี่ยน rule ได้โดยไม่ต้องแก้ core calculation เยอะ

Pattern หลักที่เหมาะสมที่สุด

สำหรับโจทย์นี้ pattern ที่สำคัญที่สุดคือ:

Strategy Pattern + Rule-based Engine + Specification Pattern

เพราะโจทย์ต้องการ:

- promotion หลายรูปแบบ
- promotion ซ้อนกัน
- เพิ่ม promotion ใหม่ในอนาคต
- ไม่กระทบ logic เดิม
- คำนวณได้ถูกต้องและตรวจสอบย้อนหลังได้

คำตอบสรุปสำหรับนำเสนอ

ระบบนี้ควรใช้ **Rule-based Promotion Engine** ร่วมกับ **Strategy Pattern**, **Specification Pattern**, **Factory/Registry Pattern** และ **Policy Pattern** เพื่อแยก logic การตรวจเงื่อนไข การเลือก promotion และการคำนวณส่วนลดออกจากกัน

Database ควรออกแบบแบบยืดหยุ่นโดยแยก promotion เป็น `promotions`, `promotion_targets`, `promotion_conditions`, `promotion_actions` และมี `promotion_calculation_logs` สำหรับตรวจสอบย้อนหลัง การออกแบบนี้ทำให้เพิ่ม promotion ใหม่ได้โดยเพิ่ม configuration ใน database และเพิ่ม Strategy เฉพาะกรณีที่ เป็น promotion type ใหม่จริงๆ โดยไม่ต้องแก้ core calculation logic เดิมมากนัก